OS & SP

Name : D. Teja Sadwika **Roll No :** 2520030536

SKILL – 16

1.Skill16.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_JOBS 100
#define MAX_CMD 256

typedef struct {
    int job_id;
    pid_t pid;
    char command[MAX_CMD];
```

```
    int running;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

/* Add a background job */
void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS) {
        printf("Job table full!\n");
        return;
    }

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;
```

```
    strncpy(jobs[job_count].command, command, MAX_CMD -
1);
    jobs[job_count].command[MAX_CMD - 1] = '\0';
    jobs[job_count].running = 1;

    printf("[%d] %d running in background\n",
           jobs[job_count].job_id,
           (int)jobs[job_count].pid);

    job_count++;
}

/* Update background job status */
void update_jobs(void)
{
    for (int i = 0; i < job_count; i++) {
        if (!jobs[i].running)
            continue;
```

```
int status;

pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

if (result == jobs[i].pid) {
    jobs[i].running = 0;

    if (WIFEXITED(status)) {
        printf("[%d] Done\t%s\n",
                jobs[i].job_id,
                jobs[i].command);
    } else if (WIFSIGNALED(status)) {
        printf("[%d] Terminated\t%s\n",
                jobs[i].job_id,
                jobs[i].command);
    }
} else if (result == -1 && errno == ECHILD) {
    jobs[i].running = 0;
```

```
        }
    }
}

/* Display job table */
void show_jobs(void)
{
    update_jobs();

    printf("\n----- Job Table -----\n");

    if (job_count == 0) {
        printf("No jobs.\n");
        return;
    }

    for (int i = 0; i < job_count; i++) {
        printf("[%d] PID=%d %s %s\n",
```

```
            jobs[i].job_id,
            (int)jobs[i].pid,
            jobs[i].running ? "Running" : "Completed",
            jobs[i].command);
    }

    printf("---------------------\n");
}

int main(void)
{
    char input[MAX_CMD];

    printf("Simple Background Job Shell\n");
    printf("Type 'exit' to quit\n");
    printf("Use '&' at the end for background execution\n");

    while (1) {
```

```
update_jobs();

printf("shell> ");
fflush(stdout);

if (fgets(input, sizeof(input), stdin) == NULL)
    break;

input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
    continue;

if (strcmp(input, "exit") == 0)
    break;

if (strcmp(input, "jobs") == 0) {
    show_jobs();
```

```
    continue;
}

int background = 0;
int len = (int)strlen(input);

if (len > 0 && input[len - 1] == '&') {
    background = 1;
    input[--len] = '\0';

    while (len > 0 &&
           (input[len - 1] == ' ' ||
            input[len - 1] == '\t')) {
        input[--len] = '\0';
    }
}

/* Keep a copy for job display */
```

```
char command_copy[MAX_CMD];

strncpy(command_copy, input, MAX_CMD - 1);
command_copy[MAX_CMD - 1] = '\0';

pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    continue;
}

/* Child process */
if (pid == 0) {
    char *args[64];
    int argc = 0;

    char *token = strtok(input, " \t");
```

```
    while (token != NULL && argc < 63) {
        args[argc++] = token;
        token = strtok(NULL, " \t");
    }

    args[argc] = NULL;

    if (argc == 0)
        _exit(0);

    execvp(args[0], args);
    perror("execvp");
    _exit(1);
}

/* Parent process */
if (background) {
```

```
        add_job(pid, command_copy);
    } else {
        int status;

        if (waitpid(pid, &status, 0) == -1)
            perror("waitpid");
    }
}

printf("\nWaiting for remaining jobs...\n");

for (int i = 0; i < job_count; i++) {
    if (jobs[i].running)
        waitpid(jobs[i].pid, NULL, 0);
}

printf("Shell terminated.\n");
```

```
    return 0;
}
```

OUTPUT :

